

Security of Distributed Software Agents in Modern Software Engineering Systems

Sarah Khaled Elshiaty (ID: S23108195), Mashael Saeed (ID: S23108689),
Seif Khaled Elshiaty (ID: S23108194)

Department of Computer Science, Effat University, Jeddah, Saudi Arabia.

Contributing authors: skelshiaty@effatuniversity.edu.sa;
mmsaeed@effatuniversity.edu.sa; skhelshiaty@effatuniversity.edu.sa;

Abstract

Distributed software agents are autonomous programs that operate across interconnected environments such as cloud computing, the Internet of Things (IoT), and artificial intelligence (AI) ecosystems. Their capacity for independent decision-making and collaboration has transformed modern computing but also introduced serious security vulnerabilities including impersonation, message tampering, and coordination failure. This study examines how contemporary software engineering (SE) methodologies can be extended to protect these agents. Using a structured literature review and analytical synthesis, it explores agent-specific security risks, engineering frameworks such as DevSecOps (Development–Security–Operations), and emerging tools like blockchain-enabled trust models. Findings reveal that although cryptographic and decentralized approaches are advancing, agent-level security remains insufficiently embedded in current SE practices. The paper concludes by advocating security-by-design principles and adaptive frameworks that combine formal verification, automation, and trust modeling for secure, scalable multi-agent systems.

Keywords: Distributed Agents, Software Security, Software Engineering Methodologies, Multi-Agent Systems, Trust Models

1 Introduction

Distributed software agents are self-governing computational entities that perform tasks across multiple systems or networks to achieve common objectives. Unlike static

modules, these agents can adapt, migrate between nodes, and collaborate dynamically, enabling applications from IoT orchestration to autonomous logistics. However, this autonomy introduces new threats. Agents can be hijacked, spoofed, or disrupted through message interception and denial-of-service attacks [1, 2]. These vulnerabilities are magnified in critical infrastructures such as healthcare and transportation, where compromised communication can cause real-world harm.

Traditional SE methodologies—Agile, DevOps, and Model-Driven Engineering (MDE)—focus on iterative improvement, automation, and modular design but often neglect the unique security demands of autonomous, decentralized systems. DevSecOps, an evolution of DevOps, integrates security testing throughout the development lifecycle but is rarely applied specifically to agent-based environments.

This paper therefore asks two guiding questions: (1) What are the primary security threats facing distributed software agents? (2) How can SE methodologies evolve to mitigate these threats effectively?

The discussion proceeds from a technical overview of distributed agents to a literature review of relevant frameworks, followed by analytical evaluation, a practical case study, and recommendations for secure agent design.

2 Technical Background

2.1 Distributed Software Agents

A distributed software agent is a self-directed program that perceives its environment, makes decisions, and acts independently or cooperatively with other agents. Core characteristics include autonomy, reactivity, cooperation, and proactivity. Depending on function, agents may be reactive (rule-based responders), cognitive (reasoning entities), or mobile (capable of migrating between systems). These capabilities drive applications in energy management, smart cities, and industrial IoT [3, 4]. However, decentralization complicates oversight. Each agent must authenticate peers and safeguard communications without relying on a central authority, creating a fertile ground for security breaches.

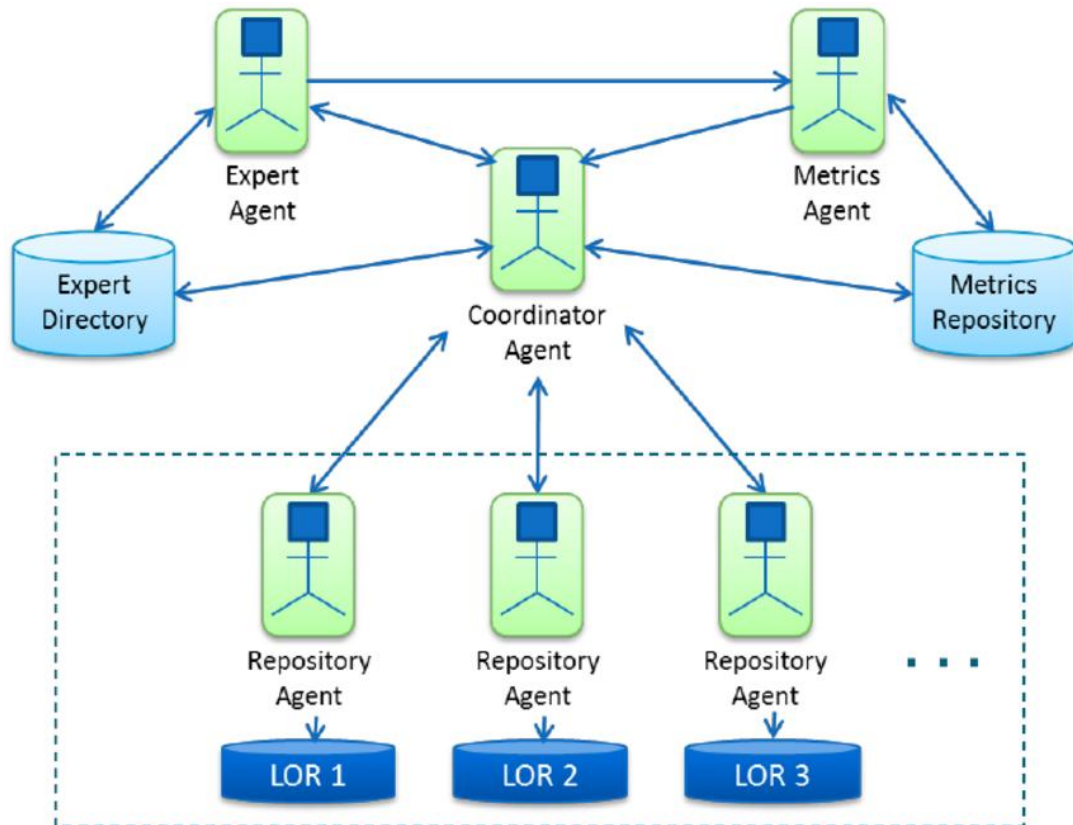


Fig. 1: Conceptual architecture of distributed software agents operating across inter- connected nodes.

2.2 Software Engineering Methodologies in Distributed Environments

Software engineering provides structured processes to manage system complexity. Agile focuses on collaboration and incremental delivery, DevOps automates integration and deployment, and DevSecOps adds embedded security testing to these workflows. MDE emphasizes abstraction and model verification prior to deployment, allowing developers to formally validate security properties of agent behavior [5]. Yet, few SE models explicitly consider the dynamic threat landscape of distributed agents. Security validation often ends at code review, leaving runtime agent interactions unchecked.

2.3 Software Security Fundamentals

Traditional security revolves around the CIA triad—Confidentiality, Integrity, and Availability. Distributed agents, however, require additional layers such as decentralized authentication, encrypted peer communication, adaptive trust models, and context-aware access control [6]. Because these systems operate without centralized management, they depend on mutual validation and lightweight cryptography—approaches that must be integrated directly into the SE process.

2.4 The Gap Between SE and Agent Security

While SE frameworks promote efficiency, they are typically designed for predictable, static systems. Distributed agents, in contrast, evolve continuously, interact unpredictably, and form transient coalitions. Without dedicated agent-security modeling, vulnerabilities such as replay attacks and rogue agent infiltration persist [7]. Bridging this gap requires merging SE rigor with decentralized verification, trust negotiation, and cryptographic resilience.

3 Literature Review

3.1 Agent-Based Software Engineering (ABSE)

Agent-Based Software Engineering (ABSE) applies SE principles to multi-agent systems. Niazi and Hussain (2017) developed a model-driven approach aligning design consistency and verifiability across distributed agents, showing that formal modeling can enhance transparency and accountability. However, they noted limited adoption of security verification at runtime. This underscores a recurring theme: agent development focuses on coordination efficiency more than defense mechanisms.

3.2 Security Threats in Distributed Agents

Studies identify impersonation, message tampering, and denial-of-service as dominant threats [8, 9]. In open networks, malicious actors can clone agents or inject false data to disrupt consensus. Real-world systems such as autonomous drone fleets or supply-chain IoT networks exemplify this risk—unauthorized interference could manipulate routes, pricing, or logistics in seconds.

3.3 Frameworks for Agent Security

Security frameworks for agents combine cryptography, authentication, and adaptive trust. Wang et al. (2022) proposed lightweight elliptic-curve cryptography suitable for IoT devices, demonstrating secure communication with minimal processing cost. Role- and attribute-based access control (RBAC, ABAC) dynamically adjust permissions, while reputation-based systems track agent reliability. Despite progress, these models face interoperability and scalability limits in multi-domain deployments [10, 11]. Integrating them within SE lifecycles can improve resilience but demands standardized APIs and cross-platform verification.

3.4 SE Methodologies Supporting Security

DevSecOps embeds automated security checks across build and deployment pipelines. Agile methods incorporate iterative threat modeling, and MDE enables formal validation of agent communication patterns [12]. Collectively, these methods strengthen security governance. However, organizational adoption lags—continuous security requires infrastructure maturity and developer expertise that many teams lack.

Table 1: Comparative overview of security techniques and their integration with SE methodologies.

Technique	Key Advantage	Limitation	SE Integration
Lightweight ECC	Low computation cost	Weak against side-channel attacks	DevSecOps
Blockchain Identity	Immutable trust	Latency issues	MDE
Reputation Systems	Dynamic trust adjustment	Fake feedback risk	Agile
ML-based IDS	Adaptive detection	Energy-intensive	DevSecOps

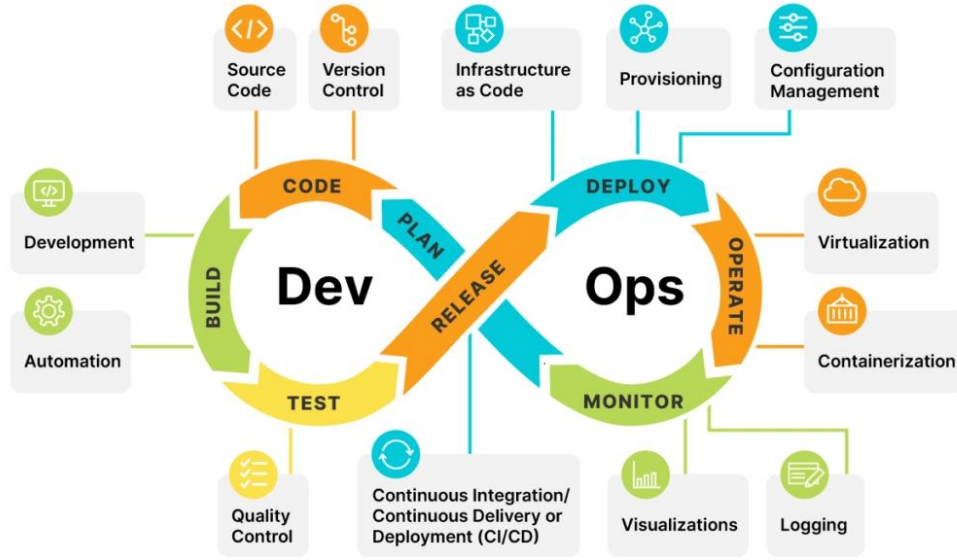


Fig. 2: Synergy between DevSecOps, MDE, and trust models ensuring secure distributed agent development.

3.5 Emerging Technologies: AI and Blockchain

Machine learning (ML) and blockchain are reshaping agent security. ML-based intrusion detection systems (IDS) recognize anomalous behaviors in agent communication, while blockchain provides immutable transaction records and decentralized identity verification [13, 14]. Combining the two allows for autonomous, self-auditing agents capable of detecting compromise in real time. Yet, energy overhead and latency remain open research problems, especially for resource-limited IoT agents.

3.6 Gaps and Challenges

Key limitations persist: cryptographic protocols increase computation costs; trust models require extensive monitoring data; and ML-based IDS struggle against zero-day attacks.

Most SE methodologies still treat security as an afterthought. Future efforts must integrate threat modeling, adaptive encryption, and verification early in the design cycle.

4 Analysis and Discussion

Integrating distributed agents into SE frameworks exposes a trade-off between flexibility and control. Cryptographic systems ensure data integrity but burden lightweight agents; reputation models foster adaptability but can be manipulated through false feedback; and AI-based IDSs enhance detection yet demand high computational power. DevSecOps offers continuous validation, Agile ensures adaptability, and MDE enforces formal structure—yet none alone fully address dynamic agent ecosystems. A hybrid methodology merging DevSecOps automation with MDE formal assurance may deliver both agility and protection.

In practice, performance remains the key constraint. For example, blockchain authentication enhances transparency but introduces latency, while lightweight agents cannot support heavy encryption. Effective SE design therefore requires context-specific trade-offs balancing performance, security, and scalability.

5 Case Study: Secure IoT Agent Framework Using Blockchain

A representative implementation appears in Roy and Zhang's (2023) blockchain-integrated multi-agent IoT framework for smart cities. Agents coordinated energy grids and traffic systems, using Hyperledger Fabric to store interactions on a tamper-proof

ledger. Smart contracts governed communication, providing decentralized authentication and accountability. Model-Driven Engineering was applied to define agent roles and constraints, while DevSecOps pipelines automated testing of smart contracts and vulnerability scans. Results showed improved integrity, resistance to denial-of-service, and transparent auditability, albeit with minor latency overheads.

This demonstrates how integrating SE methodologies with blockchain can secure distributed agents in real applications like energy management or autonomous transport. However, large-scale deployment still faces scalability and energy-efficiency challenges. Future iterations should explore lightweight consensus protocols and hybrid cloud-edge coordination to balance security with performance.

6 Conclusion and Future Work

This paper analyzed the intersection between distributed software agents and SE security. It demonstrated that while agents expand autonomy and scalability in computing, their decentralized nature introduces new vulnerabilities that traditional SE methodologies only partially address. Through literature synthesis and case analysis, the study established that integrating DevSecOps, MDE, and trust-centric frameworks can significantly strengthen resilience in agent systems.

Future research should focus on three key directions:

- Scalable blockchain protocols to reduce latency and energy consumption.
- AI-driven threat detection enabling adaptive, self-learning intrusion prevention.
- Standardized frameworks unifying agent-security modeling across SE methodologies.

By embedding these principles throughout the SE lifecycle, engineers can design distributed agents that are not only autonomous and efficient but also intrinsically secure—laying the groundwork for a trustworthy digital ecosystem.

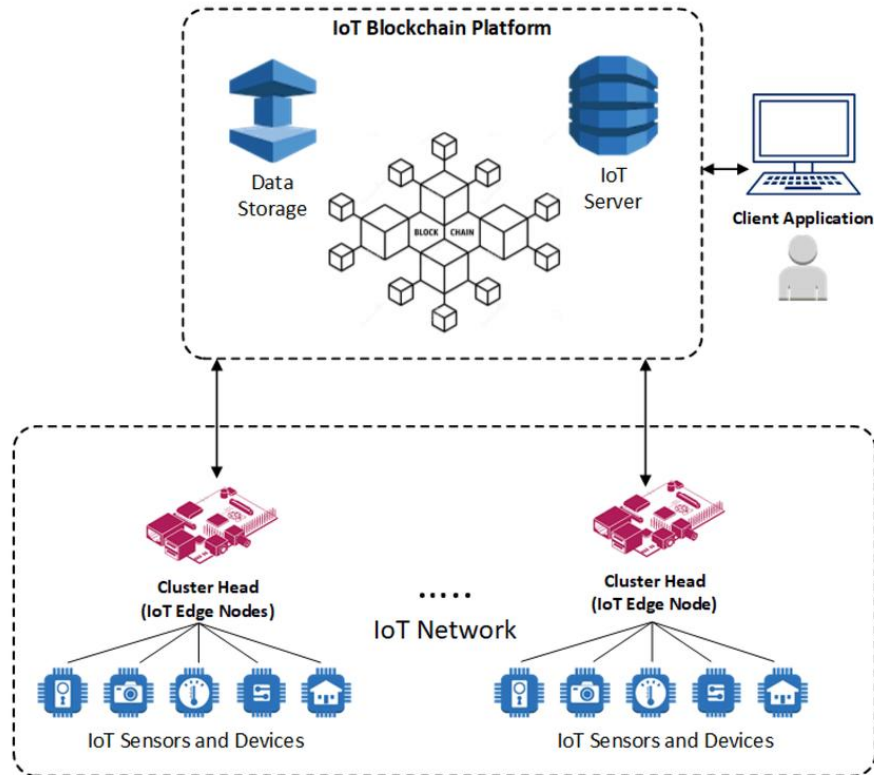


Fig. 3: Implementation workflow of the blockchain-integrated IoT agent framework.

References

- [1] Song, Y., Chen, W., Sun, X., Qin, Z., Dai, H., & Wang, K. (2025). OmniFuzz: A Multi-agent Reinforcement Learning Framework for Protocol-Aware Fuzzing in Power IoT Devices. *Journal of Network and Systems Management*, Springer. Available at: <https://link.springer.com/article/10.1007/s10922-025-09991-4>.
- [2] Pautsch, E., Singla, T., Jiang, W., & Peng, H. (2025). AgentHub: A Research Agenda for Agent Sharing Infrastructure. *arXiv preprint arXiv:2510.03495*. Available at: <https://arxiv.org/pdf/2510.03495>.
- [3] Hamad, M., Hammad, Z. A. H., & Alessi, D. (2025). Enhancing Security Through Task Migration in Software-Defined Vehicles. *IEEE Internet of Things Journal*. Available at: <https://ieeexplore.ieee.org/abstract/document/11172312>.
- [4] Khuwuthyakorn, P., Lakhan, A., & Majumdar, A. (2025). Blockchain- Enabled Self-Autonomous Intelligent Transport System for Drone Task

Workflow in Edge Cloud Networks. Algorithms (MDPI). Available at: <https://www.mdpi.com/1999-4893/18/8/530>.

- [5] Wu, J., Liu, J., Liu, Y., Wang, L., Wang, Z., & Chen, W. (2025). A Survey on Cloud-Edge-Terminal Collaborative Intelligence in AIoT Networks. arXiv preprint arXiv:2508.18803. Available at: <https://arxiv.org/pdf/2508.18803>.
- [6] Aviv, I., Taudesh, A., & Svetinovic, D. (2025). The Pref- ace Intersection of Requirements Engineering and Web3 Sys- tems. IEEE Requirements Engineering Conference. Available at: <https://ieeexplore.ieee.org/abstract/document/11190197>.
- [7] Thomas, B. (2025). Autonomous Quality Assurance: Leveraging Genera- tive AI Agents for Functional Testing of Cloud-Native Applications. Jour- nal of Computer Science and Technology Studies. Available at: <https://al-kindipublishers.org/index.php/jcsts/article/download/10367/9096>.
- [8] Prado, J. P. D. A., & Zammit Mangion, D. (2025). Enhanc- ing Flight Deck Decision Support with Distributed GenAI: A Multi-agent Approach. University of Malta. Available at: <https://www.um.edu.mt/library/oar/handle/123456789/137421>.
- [9] Santiago, R., & Goyal, S. (2025). Spring System Design in Prac- tice: Build Scalable Web Applications Using Microservices and Design Patterns in Spring and Spring Boot. IEEE Xplore. Available at: <https://ieeexplore.ieee.org/abstract/document/10989054>.
- [10] Singh, S. K. (2025). Scaling Software-Defined Networks for AI-Powered Cloud Services. International Journal of Computing and Engineering. Available at: <https://ideas.repec.org/a/bhx/ojijce/v7y2025i20p27-43id3076.html>.
- [11] Moreno-Vassart, X., & Toledo, F. J. (2025). Modeling and Analy- sis of Photovoltaic Local Energy Markets Based on Data From a Real Dataset. IEEE Transactions on Electrical Engineering. Available at: <https://ieeexplore.ieee.org/abstract/document/11169282>.
- [12] Giona, E. (2025). Quality of Service and Security in Multimodal Networks of Underwater Drones. Doctoral Thesis. Available at: https://tesidottorato.depositolegale.it/bitstream/20.500.14242/296458/4/Tesi_dottorato_giona.pdf.
- [13] Kim, Y. (2025). A Minimal Executable Framework for Decen- tralized Ethical Governance. SSRN Paper. Available at: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=5388860.
- [14] Nehzati, M. (2025). A Quantum-Inspired, Biomimetic, and Fractal Framework for Self-Healing AI Code Generation. Frontiers in Artificial Intelligence. Available at: <https://www.frontiersin.org/journals/artificial-intelligence/articles/10.3389/frai.2025.1662220/abstract>.